CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 12:
Constant Correctness and Static Functions

Course Taught by: Zubair Irshad

(For any suggested modifications please email: faisal.alvi@sse.habib.edu.pk
)

Unit Outline

1. Introduction
2. What is constant correctness?
3. Some Cases
4. Static Functions
5. Summary

Constant Correctness

- In C++, **const correctness** refers to the disciplined use of the `const` keyword to ensure that certain data or operations cannot modify an object's state after its definition.
- By enforcing immutability where appropriate, `const` helps:
  - Prevent accidental modification of values meant to remain unchanged
  - Enable better compiler optimizations and safer interfaces.
  - Code is more predictable and potential bugs are reduced.
- `const` can be applied to **variables**, **pointers**, and **member functions**

Declaring const variables

- Declaring a variable with const ensures its value **cannot be modified** after initialization.
- This helps protect important constants from accidental changes and makes code easier to reason about.
- Example:

```
const int maxSize = 100;
// maxSize = 200;
// Error: maxSize is constant and cannot be changed
```

Declaring const pointers

- In C++, const can be applied to pointers in two main ways:
- **Pointer to a Constant:** You **cannot modify the value** being pointed to, but the pointer itself can point elsewhere.

```
const int* ptr = &maxSize;
// ptr points to a constant
* ptr = 10; // Error: value cannot be changed
```

- **Constant Pointer:** You **cannot change** the pointer to point to another address, but you can modify the value it points to.

```
int value = 5;
int* const ptr = &value;
// ptr cannot point to another address
ptr = &anotherValue; // Error: pointer cannot be changed
```

Declaring const pointers

- **Hint**: Read the **const** with the value it affects i.e. the datatype etc. that comes directly after it
- **Pointer to a Constant:** You **cannot modify the value** being pointed to, but the pointer itself can point elsewhere.

```
const int* ptr = &maxSize;
// ptr points to a constant
int* ptr = 10; // Error: value cannot be changed
```

- **Constant Pointer:** You **cannot change** the pointer to point to another address, but you can modify the value it points to.

```
int value = 5;
int* const ptr = &value;
// ptr cannot point to another address
ptr = &anotherValue; // Error: pointer cannot be
```

Combining the previous 2 cases

- Constant pointer to a constant:
  - Neither the address nor the value being pointed to can be changed.

```
const int* const ptr = &maxSize;
// ptr is a constant pointer to a constant int
```

Constant Member Functions

- Declaring a member function with const ensures that the function **cannot modify** any member variables of the class.
- It enforces **object immutability** for that function and allows it to be called on **const objects**.

```
class MyClass {
    public:
        int getValue() const { return value; }
        // Constant member function – cannot modify data members

        void setValue(int newValue) { value = newValue; }
        // Non-const – can modify data members

    private:
        int value;
    };
```

Benefits of Constant Correctness

- Ensures **Integrity**: Guarantees that certain parts of your program state cannot be altered after being set.
- Enhances **Code Clarity**: Makes it clear to programmers which values are expected to remain constant.
- Enables **Optimization**: Allows compilers to perform optimizations since they know that certain values won't change.

Using constant correctness can improve both the **reliability** and **efficiency** of your C++ code.

What about `const` parameters?

- In C++, function parameters can be declared as `const` to enforce **constant correctness**.
- This guarantees that the parameter **cannot be modified** inside the function body.
- Using `const` parameters:
- It prevents **accidental changes** to input data
- `const` is most useful for **reference** and **pointer** parameters, where it protects the data being referred to.
- Example:

```
void printValue(const int& num) {
    // num cannot be modified here
    std::cout << num << std::endl;
}
```

`const` parameter with primitive types

- For **basic data types** (like `int`, `float`, or `double`), marking a parameter as `const` usually **doesn't provide much benefit**.
- That's because these parameters are **passed by value** — meaning the function receives a **copy** of the argument, not the original.

```
void printValue(const int value) {
    // value = 10;
    // Error: cannot modify a constant parameter
    std::cout << value << std::endl;
}
```

- The function still works the same with or without const, since changes inside the function **don't affect** the original variable.

`const` parameter with reference types

- For **larger objects** (like std::string, std::vector, or custom classes), **passing by reference with const** is both **efficient** and **safe**.
  - **Efficiency:** avoids making a costly copy of the object
  - **Safety:** ensures the function **cannot modify** the original object

```
void display(const std::string& message) {
// message += " - additional text";
// Error: message is constant
std::cout << message << std::endl; }
```

- Using `const` here saves the cost of copying a potentially large object while ensuring the function doesn't alter message.
- `const &` gives you the best of both worlds — **no copying** (like pass-by-reference) and **no modification** (like pass-by-value).

Constant Pointer Parameters

- When passing **pointers** to a function, marking them as const clarifies whether the function can **modify the data** being pointed to.
- **Pointer to Constant:** The **data pointed to** cannot be modified, but the **pointer itself** can be reassigned.

```
void processArray(const int* arr) {
    // *arr = 10;
    // Error: cannot modify the value pointed to
    // arr = nullptr;
    // OK: pointer itself can be changed
}
```

Constant Pointer Parameters

- **Constant Pointer Example**

```
void modifyValue(int* const ptr) {
    *ptr = 20;
    // OK: can modify the value being pointed to

    ptr = nullptr;
    // Error: pointer itself cannot be changed
}
```

- **Key Point:**
  int* const ptr → “constant pointer to int” — the pointer is fixed, but the data is not.

Constant Pointer Parameters

- **Constant Pointer to Constant:** Neither the pointer nor the data it points to can be modified.

```
void processFixedArray(const int* const arr) {
    // *arr = 10;
    // Error: cannot modify the value pointed to
    // arr = nullptr;
    // Error: cannot modify the pointer itself
}
```

Constant Parameters in Function Declarations

- In function declarations (prototypes), marking parameters as `const` helps clarify intentions for both the programmer and the compiler. For example:
- **`void compute(const int a, const std::vector<int>& data);`**

Interesting Case: **Passing Const to Non-Const Functions**

- Sometimes, `const` restrictions prevent unsafe operations — like calling a function that might modify data.

- Have a look at the following code:

```
void g1(std::string& s); // expecting non-const parameters
void f1(const std::string& s)
{
  g1(s);   // Compile-time Error since s is const
  std::string localCopy = s; //creates a deep copy
  g1(localCopy); // Okay since localCopy is not const
}
```

- **Key Takeaways:**
  - You **cannot pass a const object** to a function expecting a **non-const reference**.
  - To modify such data, make a **copy** first — preserving const-correctness while still allowing changes when needed.

Some Questions – General form!

- What does "`const X* p`" mean?
- What does "`const X& x`" mean?
- Other interesting questions with answers here: https://isocpp.org/wiki/faq/const-correctness#overview-const

.

  - For instance:
    - **What's the difference between "`const X* p`", "`X* const p`" and "`const X* const p`"?**
    - **Does "`X& const x`" make any sense?**

What does "`const X* p`" mean?

- It means p points to an object of class X, but p can't be used to change that X object (naturally p could also be NULL).
- Read it right-to-left: "**p is a pointer to an X that is constant.**"
- For example, if class X has a const member function such as inspect() const, it is okay to say **p->inspect().** But if class X has a non-constant member function called mutate(), it is an **error** if you say **p->mutate()**.

What does “`const X& x`” mean?

- It means x aliases an X object, but you can’t change that X object via x.
- Read it right-to-left: “x is a reference to an X that is const.”
- For example, if class X has a const member function such as **inspect()** const, it is okay to say x.inspect(). But if class X has a non-const member function called **mutate(),** it is an **error** if you say x.mutate().

Static Variables – Review

- Recall that we have studied static variables earlier in the course.
- A static variable (also called a class variable) maintains only one copy for the entire class.
- In other words, each object of a class does not have a separate copy of static variable.
- Instead all objects of the class share the same copy of the static variable.

Static Functions

- In C++, a **static function** is a function that belongs to a **class** but **does not operate on a specific instance of that class**.
- Static functions are declared with the **`static`** keyword and have unique properties and use cases.

Properties of a Static Function

- Static functions **belong to the class itself**, **not to any particular object**.
- Remember when calling functions, we usually call it with the object of the class!
- Static function called directly on the class without creating an instance of the class.

```
class MathUtils {
public:
    static int add(int a, int b) {
        return a + b;
    }
};
int result = MathUtils::add(5, 3);
// No object needed to call add()
```

Access Restrictions

- Static functions can **only access** other static members (vars/functions) of the class.
- They do not have access to **`this`**, as they do not belong to any specific instance.

```
class Counter {
private:
    static int count; // Static member variable
public:
    static void increment() {
        count++; // Can access static variable
        // nonStaticVariable++;
       // Error: cannot access non-static members
    }};

int Counter::count = 0; // Initialize static member outside class
```

Cannot be Virtual

- Static functions **cannot be virtual**, meaning they **do not support polymorphism or dynamic dispatch**.
- This is because **virtual functions work with instances of a class**, **while static functions are tied to the class itself**.

Use Cases

- Static functions **are often used for utility functions** that don't need an object to work.
- Common examples include **mathematical calculations**, **logging functions**.

```
class Logger {
public:
    static void log(const std::string& message) {
        std::cout << "Log: " << message << std::endl;
    }
};
Logger::log("This is a message.");
// Can call log without a Logger object
```

Static Functions – Benefits

- Static functions are useful when a **function's logic is specific to the class** but doesn't need to interact with the instance's state.
- This can help **encapsulate logic within the class itself**, rather than **defining the function globally**.
- Static functions have a **lifetime** independent of any class instances.
- They exist as long as the program runs, and they don't require any instance initialization to be accessible.

Example: `Factory` design pattern

- A common use of static functions is in **implementing the Factory Design pattern**. (We will learn about design patterns in future classes)
- Here, a static function creates instances of a class without requiring an instance of that class.

```
class Product {
public:
    static Product createDefault() {
        // Create a default Product instance
        return Product();
    }};
 Product defaultProduct = Product::createDefault();
// Create a Product instance using static factory method
```

- Encapsulates object creation logic inside the class
- Avoids exposing constructors directly

Summary

- Const correctness is an important aspect of type correctness in C++.
- There are several variations of const correctness, each with its own intricacies.
- Static functions, just like static variables, are class functions.
- Static functions are not tied to specific objects. Rather they are linked with the class instances.